AD-SFL: Activation-Space Anomaly Detection and Reinforcement Learning-Based Defense Against Data Poisoning in SplitFed Learning

Tasnimul Hossain Tomal[a], Intesar Tahmid[a], Palash Roy[a], Md. Abdur Razzaque[a], Md Rafiul Hassan[b], Mohammad Mehedi Hassan[c]

[a]*Green Networking Research Group, Department of Computer Science and Engineering, University of Dhaka, Dhaka - 1000, Bangladesh*
[b]*Department of Computer Science, Central Connecticut State University, New Britain, CT, USA*
[c]*Information Systems Department, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia*

Abstract

SplitFed Learning (SFL) is an emerging distributed training paradigm that combines the communication efficiency of split learning with the collaborative optimization of federated learning, enabling privacy-preserving model training on resource-constrained clients. However, the architectural separation between client and server introduces new security vulnerabilities, particularly data poisoning attacks such as label flipping and backdoor injection. Existing defenses designed for federated learning rely on inspecting client model updates or gradients. However, these are not accessible in SplitFed environments, which share only intermediate activations, thereby leaving a critical robustness gap. This paper proposes AD-SFL, an activation-space defense framework that detects and mitigates data poisoning attacks by analyzing the statistical divergence between client activation distributions and a trusted reference distribution. The framework integrates three key components: activation-based anomaly detection using KL-divergence, temporal reputation modeling based on subjective logic to track client trustworthiness over time, and adaptive client selection through a Deep Deterministic Policy Gradient (DDPG) reinforcement learning agent. This combination enables robust and adaptive filtering of malicious clients without requiring access to raw data or model parameters. Extensive experiments on MNIST and CIFAR-10 under label-flipping and backdoor attack scenarios demonstrate that AD-SFL significantly improves robustness compared to existing baselines. The proposed framework maintains high model utility while effectively suppressing attack success rates, even with a high proportion of malicious clients. Specifically, AD-SFL achieves up to 96.26% accuracy with a 1.93% attack success rate on MNIST, and 72.70% accuracy with a 1.67% attack success rate on CIFAR-10.

Keywords: SplitFed Learning, Data Poisoning, Activation Space, Deep Reinforcement Learning Control, Temporal Reputation Modeling

1. Introduction

The growing adoption of edge intelligence and privacy regulations has driven wide interest in distributed learning paradigms, notably Federated Learning (FL) [1] and Split Learning (SL) [2]. SplitFed Learning is a hybrid architecture that combines the heterogeneity handling capability of FL with the computational efficiency of SL [3]. This architecture effectively overcomes the limitations of traditional distributed learning by enabling training on low-computation devices while strictly enforcing data localization. Consequently, SplitFed is positioned to become a core enabler for future hybrid infrastructures, such as decentralized IoT and medical systems, where full-model sharing is often prohibited. For example, in the Internet of Medical Things (IoMT), ultra-low-power implantable devices cannot perform full neural network backpropagation for FL without draining battery life and posing physical risks. Conversely, transmitting raw physiological data sequentially for SL violates privacy regulations like HIPAA and introduces latency. Similarly, in Industrial Control Systems (ICS), legacy sensors monitoring power grids lack the capacity for full model parameter storage, yet their sensitive operational data cannot be centralized. In these scenarios, SplitFed bridges the gap by offloading computational workloads to the server via "smashed" activations while processing data in parallel.

Despite its architectural advantages, SplitFed learning introduces a critical vulnerability: a single compromised node can inject poisoned data that aggregates into the global model, making robust defense mechanisms a prerequisite for real-world deployment. While FL and SL advance distributed privacy, they struggle to meet the demands of securing critical infrastructures. Because SplitFed is uniquely positioned as the enabling framework for high-stakes, privacy-critical applications such as IoMT and industrial control systems, a silently corrupted global model directly translates into catastrophic real-world failures, such as misdiagnosing patient anomalies or ignoring critical infrastructure faults. Addressing this challenge is difficult because the system requires an online, adaptive defense that operates exclusively on server-observable signals (activations) while remaining stable under non-IID data heterogeneity.

The motivation to secure SplitFed learning stems from the severe inadequacy of current distributed security mechanisms when applied to this unique hybrid framework. Deploying standard FL defenses in a SplitFed environment is fundamentally

Email addresses: mdrafiul.hassan@ccsu.edu (Md Rafiul Hassan), mmhassan@Ksu.edu.sa (Mohammad Mehedi Hassan)

flawed and introduces a false sense of security. Specifically, existing Byzantine-robust aggregation protocols and similarity-based heuristics, such as Krum, Trimmed Mean, and FoolsGold [4, 5, 6], were exclusively designed for gradient- or parameter-space interactions. They strictly rely on inspecting the statistical distribution of full model updates (ΔW) to identify malicious outliers. However, SplitFed enforces a blind server architecture: the server never receives the clients' local weights or gradients, making existing FL defenses mathematically inapplicable since the server cannot compute Euclidean distances or cosine similarities between client models it cannot observe. This architectural blind spot motivates our work; without a bespoke defense, an attacker can freely poison the local feature extractor to flip labels or inject backdoors, leaving standard aggregation rules completely oblivious.

To address this critical practical gap, this paper tackles the detection and mitigation of data poisoning in SplitFed, focusing on two practically relevant families: (i) *label-flipping* attacks that degrade global utility by systematically corrupting class labels, and (ii) *backdoor-style* poisoning that implants trigger-based misclassification while preserving apparent clean accuracy [7]. We consider a strong yet realistic threat model in which a fixed fraction of clients may intermittently behave maliciously. To address this critical practical gap, this paper tackles the detection and mitigation of data poisoning in SplitFed, focusing on two practically relevant families: (i) *label-flipping* attacks that degrade global utility by systematically corrupting class labels, and (ii) *backdoor-style* poisoning that implants trigger-based misclassification while preserving apparent clean accuracy [7]. We consider a strong yet realistic threat model in which a fixed fraction of clients may intermittently behave maliciously. To overcome this, we propose AD-SFL, a novel reputation-based mechanism that utilizes the activation space in the smashed data to detect malicious clients. To the best of our knowledge, no prior work provides activation-space anomaly detection combined with adaptive RL-based thresholding in SFL. Our methodology involves three core components:

- **Activation-centric poisoning detection for SplitFed:** We achieve this by analyzing class-conditional smashed activation distributions and quantifying client deviations via KL divergence, without requiring access to raw client data.
- **Temporal reputation modeling with subjective logic:** We convert per-round anomaly evidence into *time-evolving* client reputations using subjective-logic opinions, enabling robust aggregation decisions that incorporate both short-term deviations and long-term behavioral consistency.
- **Adaptive client selection through learned threshold control:** We cast reputation thresholding as a continuous-control problem and learn a dynamic acceptance threshold using DDPG, allowing AD-SFL to adapt to changing attack intensity, client heterogeneity, and training progress.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature on several distributed learning paradigms, data poisoning attacks, and robust aggregation defenses. Section 3 defines the system environment, detailing the threat model, attacker capabilities, and privacy assumptions. Section 4 introduces the AD-SFL architecture, elaborating on the activation-based anomaly detection, subjective logic reputation updates, and the reinforcement learning-based threshold adaptation. Section 5 presents extensive experimental results on MNIST and CIFAR-10, analyzing model robustness, detection performance, and system adaptivity. Finally, Section 6 concludes the work and suggests future research directions.

2. Related Work

The domains of Federated Learning (FL) and Split Learning (SL) have witnessed significant advancements, yet securing these distributed frameworks against adversarial manipulation remains a critical challenge.

2.1. Data Poisoning Defenses in Federated Learning and Limitations

Data poisoning remains a primary threat to distributed learning, in which adversaries degrade the global model's utility via label-flipping or backdoor injections [7, 16]. In the context of FL, numerous robust aggregation rules have been proposed. Classic Byzantine-resilient mechanisms like KRUM [4] and Trimmed Mean [5] filter malicious updates based on Euclidean distance or statistical outliers. More recent approach like FLGuardian [8] introduces a fine-grained layer-wise detection mechanism using pairwise cosine distances to identify benign clients. Similarly, PDFL [17] leverages secure multi-party computation to perform cluster analysis on encrypted gradients, effectively identifying poisoning attempts. DefendFL [9] utilizes mask-based gradient concealment combined with cosine similarity checks to detect malicious gradients without exposing raw updates.

However, while effective in standard FL, these defenses share a fundamental limitation when applied to SplitFed Learning: they rely on inspecting *full model updates* or *gradients*. In SplitFed, the server is architecturally "blind" to the client-side subnetwork and only receives smashed activations. Consequently, defenses that require computing distances between model weights (like KRUM or FLGuardian) are mathematically inapplicable in the SplitFed context.

2.2. Security Vulnerabilities in Split and SplitFed Learning

Security research in SplitFed learning is still limited. The architecture's unique split nature introduces distinct vulnerabilities. Ying et al. introduce a label-blind backdoor attack on vertical SplitNNs [18], while SBPA leverages Sybil nodes to amplify backdoor poisoning in AIoT systems [19]. Recent works have attempted to bridge this gap. RoFed-LLM [15] proposes a robust framework for Large Language Models by integrating split federated learning with adaptive jamming defenses to mitigate wireless adversarial attacks. While defenses like SafeSplit [13] and ZORRO [14] introduce mechanisms to prevent backdoor injections, they are designed primarily for the sequential training paradigm of standard SL.

Method / Defense	Target Architecture	Operating Space	Mitigates Poisoning?	Handles Non-IID?	Temporal Reputation?
KRUM [4]	FL	Weights / Gradients	Yes	No	No
Trimmed Mean [5]	FL	Weights / Gradients	Yes	No	No
FLGuardian [8]	FL	Weights / Gradients	Yes	No	No
DefendFL [9]	FL	Weights / Gradients	Yes	No	No
HeteroFL [10]	FL	Weights / Gradients	Yes	Yes	No
FedCoguard [11]	FL	Weights / Gradients	Yes	Yes	No
FORTRESS [12]	FL	Weights / Gradients	Yes	Partial	Yes (DRL)
SafeSplit [13]	SL	Server Backbone Weights	Backdoor Only	No	No
ZORRO [14]	SL	Client Weights (via ZKP)	Backdoor Only	No	No
RoFed-LLM [15]	SFL	Signals / Weights	No (Jamming)	No	No
AD-SFL (Proposed)	**SFL**	**Smashed Activations**	**Yes**	**Yes**	**Yes (DRL)**

Table 1: Comparison of AD-SFL with existing distributed learning defense mechanisms.

Critically, existing SplitFed defenses often focus on privacy leakage rather than poisoning robustness. While RoFed-LLM addresses external wireless attacks, it does not explicitly tackle internal data poisoning, where valid authenticated clients submit corrupted data. Furthermore, most split-learning defenses operate on the assumption of sequential training, which does not scale to the parallel aggregation inherent in SplitFed.

2.3. The Challenge of Non-IID Data

A major hurdle in detecting poisoning is distinguishing between malicious behavior and benign statistical deviation caused by Non-Independent and Identically Distributed (Non-IID) data. Consequently, skewed distributions frequently cause honest clients to trigger false positives. HeteroFL [10] addresses this by employing a succession of defensive strategies specifically tailored for Non-IID FL, ensuring the model converges even with unbalanced data. Similarly, FedCoguard [11] proposes a client-side defense for agricultural IoT that leverages intrinsic training objective differences to filter malicious updates in highly Non-IID settings. LFR-PPFL [20] utilizes temporal cosine similarity analysis to distinguish label-flipping attacks from Non-IID data variations.

Although these methods improve robustness against heterogeneity, they are designed for the parameter space of FL. In SplitFed, Non-IID data manifests as shifts in the *activation distributions* rather than weight distributions. Because existing defenses ignore the statistical properties of smashed data, AD-SFL is specifically tailored to handle heterogeneity directly in this activation space.

2.4. Trust and Reputation Mechanisms

Reputation-based systems offer a dynamic alternative to static rejection rules by tracking client behavior over time. The foundational work in this domain introduced Differentially Private SGD (DP-SGD) to bound privacy loss [21], while recent works like Maslamani et al. employed Deep Reinforcement Learning (DRL) to dynamically filter malicious workers [22]. FORTRESS [12] represents the state-of-the-art in this domain, utilizing an encoder-decoder architecture driven by Actor-Critic (AC) reinforcement learning to assess client trust based on dual-domain distance metrics. Similarly, PFLSE [23] introduces a Shannon entropy-based metric to weigh client contributions, allowing models with richer information to carry more influence. Contract-theoretic approaches construct incentive compatible reputations to penalize malicious behavior over time [24], while similarity-based defenses like FoolsGold [6] mitigate Sybil attacks.

While promising, these approaches have limitations in the SplitFed regime. FORTRESS relies on dual-domain (weight and gradient) metrics, which are unavailable to a SplitFed server. PFLSE, while using entropy, focuses on weighted aggregation rather than the binary exclusion of malicious actors needed to stop poisoning. AD-SFL advances the state of the art by adapting reinforcement learning and entropy-based analysis to the activation space observable in SplitFed.

3. System Environment and Assumptions

3.1. Threat Model and Attack Assumptions

3.1.1. Data Poisoning Attack Framework

We consider the SplitFed setting of Section 4.1 with N clients $C = \{C_1, \ldots, C_N\}$ and one server. A fixed fraction $f_m \in (0, 1)$ of clients is adversarial, forming the malicious set $\mathcal{M} \subset C$; all remaining clients are honest. Malicious clients perform local data-poisoning in each round $t \in \{1, \ldots, T\}$ by modifying samples in their mini-batches following standard federated threat models [16]. We study two attack types.

Label flipping. The attacker selects a set of targeted label pairs $\mathcal{P}$. For any sample whose label belongs to a targeted pair, the label is swapped to its paired class:

$$y'_{i,j} = \begin{cases} \pi(y_{i,j}), & \text{if } y_{i,j} \in \bigcup_{(a,b)\in\mathcal{P}}\{a, b\}, \\ y_{i,j}, & \text{otherwise,} \end{cases} \tag{1}$$

where $\pi(\cdot)$ maps each label to its paired counterpart (and leaves all other labels unchanged).

Backdoor poisoning. The attacker implants a trigger-based misclassification by (i) stamping a fixed trigger via an input transformation $\Delta(\cdot)$ and (ii) relabeling poisoned samples to a single

target class $y^\star$. For source labels $\mathcal{S}$, all such samples are poisoned:

$$(\mathbf{x}'_{i,j}, y'_{i,j}) = \begin{cases} (\Delta(\mathbf{x}_{i,j}),\ y^\star), & \text{if } y_{i,j} \in \mathcal{S}, \\ (\mathbf{x}_{i,j},\ y_{i,j}), & \text{otherwise.} \end{cases} \tag{2}$$

SplitFed interaction. The model is split into client- and server-side subnetworks $f_C(\cdot; W_C)$ and $f_S(\cdot; W_S)$. For the stacked batch $\mathbf{X}_i^t$, the client computes smashed activations

$$h_{i,t} = f_C(\mathbf{X}_i^t; W_C^t), \tag{3}$$

and sends $(h_{i,t}, Y_i^t)$ to the server, where Y_i^t are clean labels for honest clients and poisoned labels (from (1) or (2)) for malicious clients. Both attacks disrupt class-conditional activation statistics, which AD-SFL leverages as an anomaly signal.

3.1.2. Attacker Capabilities and Constraints

We assume a relatively strong but realistic adversary. Each malicious client $C_i \in \mathcal{M}$ has full control over the labels of its local training data and may choose which samples $(\mathbf{x}_{i,j}, y_{i,j})$ to corrupt. Malicious clients are also granted white-box knowledge of the architecture of the global client-side subnetwork $f_C(\cdot)$, as well as the training hyperparameters (e.g., learning rates, batch sizes, and number of global rounds T) and the overall SplitFed training protocol.

At the same time, the adversary is constrained by standard privacy and network-security assumptions in SplitFed deployments. Malicious clients cannot access raw data, smashed activations, or model states of other clients; their view is limited to their own local data, mini-batches, and the model parameters broadcast by the server, i.e., the shared client-side weights W_C^t and server-side weights W_S^t at the beginning of each round. They are unable to intercept, reorder, or modify server-client communications, which are assumed to be protected by transport-layer security, and cannot tamper with the server's internal state.

Let $\mathcal{H}_t \subseteq \{1, \ldots, N\}$ denote the set of clients accepted for aggregation in round t. Once a client C_i is flagged as malicious by the reputation-based filtering mechanism in AD-SFL, its index i is excluded from $\mathcal{H}_t$, and its update does not contribute to the aggregation in that global round.

3.2. System Assumptions

3.2.1. Reference Dataset Assumption

AD-SFL assumes the existence of a small, clean reference dataset

$$\mathcal{D}_{\text{ref}} = \bigcup_{k=1}^{K} \mathcal{D}_{\text{ref}}^{(k)}$$

stored on the server, where K is the number of classes and $\mathcal{D}_{\text{ref}}^{(k)}$ denotes the subset of reference samples labeled as class k. This dataset is guaranteed to be uncontaminated and is used to construct a stable reference for honest behaviour in the activation space.

In practice, $\mathcal{D}_{\text{ref}}$ can be obtained from public benchmarks, internal validation data collected under controlled conditions, or small, curated samples contributed by clients during system initialization and carefully checked for label accuracy. The total size of $\mathcal{D}_{\text{ref}}$ does not need to scale with the number of clients N and can be modest (e.g., on the order of a handful of examples per class), since it is used only to estimate class-conditional reference densities $\hat{p}_{\text{ref},t}^{(k)}$ rather than to train the main model.

3.2.2. Heterogeneous Data Distribution

We explicitly assume that client data are heterogeneous and non-IID, reflecting real-world SplitFed deployments. Formally, each client C_i has its own local dataset $\mathcal{D}_i$ drawn from a client-specific distribution, which may differ significantly across i. In particular, different clients may focus on different subsets of classes, exhibit strong class imbalance, or experience distribution shifts over time and across regions; they may also contribute very different mini-batch sizes n_i in each round t.

AD-SFL is designed to be robust under this heterogeneity. For each round t and client C_i, the activation-based anomaly detector produces an anomaly score δ_i^t using (11), and these scores are normalized across clients to obtain $\hat{\delta}_i^t$ as in (12). AD-SFL makes its anomaly detection adaptive to the current mix of clients and reduces the risk of spuriously flagging clients whose data legitimately differ from the reference distribution.

3.2.3. Communication and Computational Constraints

We consider a communication-efficient SplitFed training setting in which clients transmit only smashed activations at the cut layer and, when accepted, their updated client-side parameters to the server, in line with the workflow in Algorithms 1 and 2. Raw input samples, full layer-wise gradients, and internal server-side activations are never communicated over the network. To respect the limited computational resources of edge devices, AD-SFL delegates all additional work introduced by anomaly detection and adaptive thresholding to the server.

3.3. Privacy Preservation

AD-SFL is designed to preserve the privacy guarantees already offered by SplitFed learning. All anomaly detection logic and reinforcement-learning-based threshold adaptation operate solely on the server and do not require any additional client information beyond what the baseline protocol transmits. In particular, the server relies only on quantities already present in SplitFed, namely the smashed activations $\mathbf{a}_{i,t}$, per-client losses $\mathcal{L}_i^t$, and the aggregated anomaly and reputation scores $(\delta_i^t, \hat{\delta}_i^t, \gamma_i^t)$. We further assume that the central server is *honest and non-curious*: it executes the AD-SFL protocol correctly and does not attempt to infer extra information about client data from the received activations or losses. Under this assumption, AD-SFL does not introduce new privacy attack surfaces for clients.

Prior work shows that intermediate activations can, in principle, leak sensitive information through reconstruction or inversion attacks [25, 26]. These risks arise from the underlying SplitFed protocol and are orthogonal to AD-SFL itself. They can be mitigated through complementary defences such as adding differential privacy noise to activations, employing secure multi-party computation for server-side processing, or executing the server within trusted execution environments. Such techniques

can be layered on top of AD-SFL without altering its core design or its notation.

4. Design Details of AD-SFL

AD-SFL is designed to detect label-flip and other data poisoning attacks in SplitFed learning by analysing client-side smashed activations. This section specifies the system model and shows how KL-divergence-based activation anomaly detection, Fisher-based thresholding and DDPG-based control are instantiated inside the SplitFed training loop.

4.1. SplitFed Learning Model

We consider a SplitFed learning system with N clients and a central server. The global model is decomposed into a client-side subnetwork $f_C(\cdot)$ and a server-side subnetwork $f_S(\cdot)$, parametrized by W_C and W_S, respectively. Training proceeds in communication rounds indexed by $t \in \{1, \ldots, T\}$, where W_C^t and W_S^t denote the client-side and server-side parameters at the beginning of round t. Unlike the original SplitFed formulation that separates responsibilities between a *main server* (handling forward/backward on f_S) and a *federated aggregation server* (aggregating client-side updates), we consider a unified design in which the main server performs *both* roles for simplicity. This consolidation also avoids an additional coordination/latency path that would otherwise couple anomaly decisions to aggregation. This design, however, does not restrict generality; the framework transfers directly to the two-server SplitFed architecture by having the main server communicate the indices of accepted/honest clients in round t to the federated aggregation server, which then aggregates only those client-side updates.

For a participating client i in round t, let

$$\mathcal{B}_i^t = \{(\mathbf{x}_{i,j}, y_{i,j})\}_{j=1}^{n_i}$$

denote a mini-batch sampled from its local dataset $\mathcal{D}_i$, where n_i is the batch size. Stacking the inputs as

$$\mathbf{X}_i^t = [\mathbf{x}_{i,1}, \ldots, \mathbf{x}_{i,n_i}],$$

The split forward pass is

$$\mathbf{a}_{i,t} = f_C(\mathbf{X}_i^t; W_C^t), \tag{4}$$

$$\hat{\mathbf{y}}_{i,t} = f_S(\mathbf{a}_{i,t}; W_S^t), \tag{5}$$

where $\mathbf{a}_{i,t}$ are the smashed activations transmitted from client i to the server in round t, and $\hat{\mathbf{y}}_{i,t}$ are the corresponding server-side predictions. Because $\mathbf{a}_{i,t}$ is already communicated to complete the split forward pass, it provides a natural observation point for attack monitoring without requesting additional client-side telemetry or raw data exposure.

The server computes a per-client cross-entropy loss

$$\mathcal{L}_i^t = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell\Big(\hat{y}_{i,j}^t, y_{i,j}\Big), \tag{6}$$

where $\hat{y}_{i,j}^t$ is the predicted output for sample $\mathbf{x}_{i,j}$ and $y_{i,j}$ is its ground-truth label.

4.2. Anomaly Detection via KL Divergence

AD-SFL treats client-side smashed activations as class conditional random variables and detects malicious clients by measuring how their activation distributions deviate from a clean reference. A distributional criterion is especially suitable for label-flip/poisoning, which tends to induce systematic shape and support shifts in class-conditional representations rather than isolated outliers.

4.2.1. KL Divergence and Sample-Based Estimation

Given densities p and q on a continuous domain $\mathcal{X}$, the Kullback–Leibler (KL) divergence is

$$D_{\mathrm{KL}}(p\|q) = \int_{x\in\mathcal{X}} p(x) \log \frac{p(x)}{q(x)}\, dx. \tag{7}$$

Using the identity $D_{\mathrm{KL}}(p\|q) = \mathbb{E}_{x\sim p}[\log(p(x)/q(x))]$, a Monte Carlo approximation from samples $\{x_m\}_{m=1}^M \sim p$ is

$$D_{\mathrm{KL}}(p\|q) \approx \frac{1}{M} \sum_{m=1}^{M} \log \frac{\hat{p}(x_m)}{\hat{q}(x_m)}. \tag{8}$$

This sample-based form aligns with the minibatch-driven SplitFed loop, allowing divergence estimates to be computed online from the same activations already produced for training.

4.2.2. Kernel Density Estimation

Because the true densities are not available, AD-SFL uses kernel density estimation (KDE) to fit $\hat{p}$ and $\hat{q}$ from samples. KDE avoids imposing a restrictive parametric form on smashed-activation statistics, which can be heavy-tailed and layer-dependent as the representation evolves across rounds. For scalar samples $\{x_m\}_{m=1}^M$, KDE is

$$\hat{p}(x) = \frac{1}{Mh} \sum_{m=1}^{M} K\Big(\frac{x - x_m}{h}\Big), \tag{9}$$

where $K(\cdot)$ is a Gaussian kernel. The bandwidth h is chosen by Silverman's rule of thumb [27]:

$$h = 0.9 \min\Big(\sigma, \frac{\mathrm{IQR}}{1.34}\Big) M^{-1/5}. \tag{10}$$

Silverman's rule provides a stable, data-adaptive bandwidth that reduces per-dataset tuning, which is desirable when the detector must track a nonstationary feature space over training.

4.2.3. Class-conditional Activations

For client i in round t and class $k \in \{1, \ldots, K\}$, we define the index set of class-k samples in the mini-batch as

$$\mathcal{I}_k^{i,t} = \{j : y_{i,j} = k\}.$$

Conditioning on k sharpens sensitivity to label flips, since the attack's signature is precisely a mismatch between the label index and the representation that the client emits for that class. The corresponding smashed activations are

$$\mathbf{a}_{i,j}^t = f_C(\mathbf{x}_{i,j}; W_C^t), \quad j \in \mathcal{I}_k^{i,t}.$$

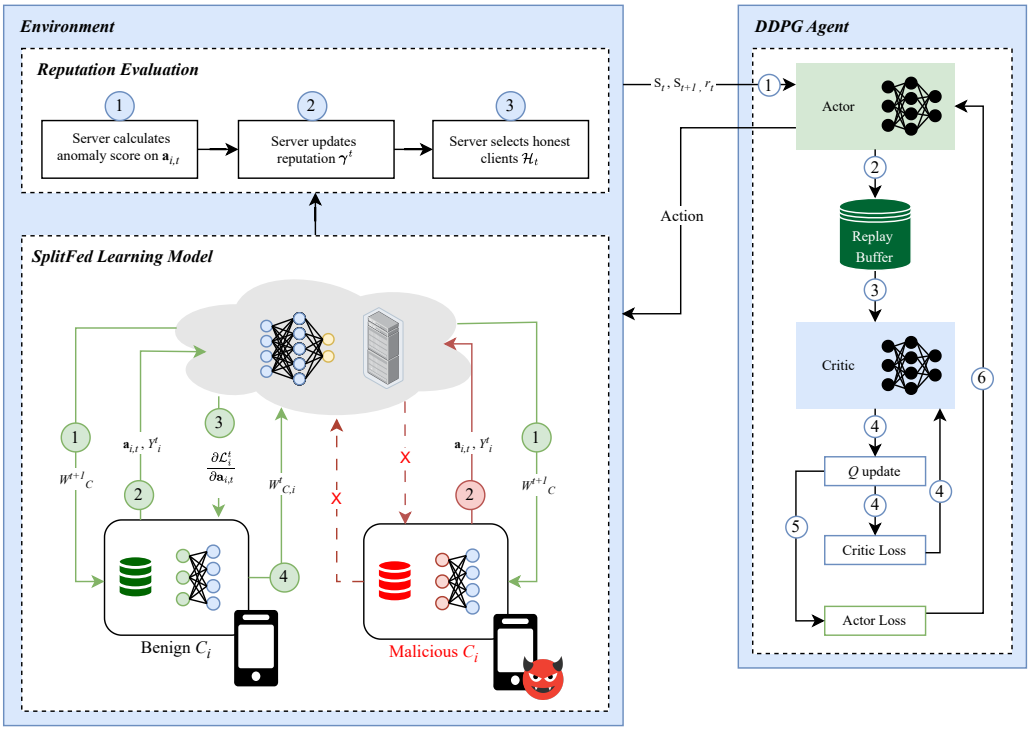


Figure 1: System model of the proposed AD-SFL framework.

We flatten all activations of class k into a single vector

$$\mathbf{v}_{i,t}^{(k)} = \text{Flatten}\Big(\{\mathbf{a}_{i,j}^{t} : j \in \mathcal{I}_k^{i,t}\}\Big),$$

which we interpret as $M_i^{(k,t)}$ scalar samples from a client-specific distribution for class k. Flattening yields a large set of scalar samples per class from a single minibatch, making density estimation numerically stable at low communication cost, while still reflecting systematic representation drift induced by poisoning.

4.2.4. Clean Reference Distributions

The server maintains a small, trusted reference dataset $\mathcal{D}_{\text{ref}}$, assumed to be free of attacks. A compact trusted reference is sufficient because the detector needs a stable anchor for relative deviation rather than exhaustive coverage, and its activations can be recomputed under the current W_C^t to remain aligned with the evolving representation. For each class k, the subset $\mathcal{D}_{\text{ref}}^{(k)}$ is passed through the current client-side network to obtain reference activations

$$\mathbf{a}_{\text{ref}}^{(\ell,k,t)} = f_C(\mathbf{x}_{\text{ref}}^{(\ell,k)}; W_C^t), \quad (\mathbf{x}_{\text{ref}}^{(\ell,k)}, k) \in \mathcal{D}_{\text{ref}}^{(k)},$$

which are flattened into $\mathbf{v}_{\text{ref},t}^{(k)}$. For each class k, we estimate the reference density $\hat{p}_{\text{ref},t}^{(k)}$ and the client-specific density $\hat{p}_{i,t}^{(k)}$ using KDE with Gaussian kernels and bandwidth h from (10).

4.2.5. Per-class and Aggregated Anomaly Scores

Using (8), we compute a per-class divergence

$$D_{\text{KL}}^{(k)}(i,t) \approx D_{\text{KL}}(\hat{p}_{i,t}^{(k)} \,\|\, \hat{p}_{\text{ref},t}^{(k)}),$$

for each class k observed in client i's mini-batch. Defining

$$\mathcal{K}_i^t = \{k : |\mathcal{I}_k^{i,t}| > 0\},$$

and letting the class weight be proportional to the client's class mass for non-iid client handling

$$w_{i,t}^{(k)} = \frac{|\mathcal{I}_k^{i,t}|}{\sum_{j\in\mathcal{K}_i^t} |\mathcal{I}_j^{i,t}|}, \qquad \sum_{k\in\mathcal{K}_i^t} w_{i,t}^{(k)} = 1,$$

the aggregate anomaly score for client i at round t is

$$\delta_i^t = \sum_{k\in\mathcal{K}_i^t} w_{i,t}^{(k)} D_{\text{KL}}^{(k)}(i,t). \tag{11}$$

The weighting prevents clients with sparse incidental class occurrences from dominating the aggregate score and keeps δ_i^t comparable under heterogeneous label distributions. A higher δ_i^t indicates stronger deviation from clean reference behaviour, which is sensitive to systematic label flips that distort the alignment between activations and class labels.

For comparability across clients, we normalize:

$$\hat{\delta}_i^t = \frac{\delta_i^t}{\sum_{j=1}^{N} \delta_j^t + \epsilon}, \tag{12}$$

where $\epsilon > 0$ is a small constant for numerical stability. This normalization reduces sensitivity to round-to-round scale fluctuations (e.g., early-training instability) and provides a consistent anomaly signal for downstream thresholding and reputation updates. The normalized scores $\{\hat{\delta}_i^t\}_{i=1}^N$ are the anomaly signals consumed by the reputation and thresholding modules.

4.3. Reputation Modelling

AD-SFL maps anomaly evidence into a trust score using a subjective logic-based mechanism [28]. Each client's reputation is a time-evolving opinion that aggregates past and current behaviour rather than relying on a single round. This evidence-based smoothing is beneficial in SplitFed because minibatch-level anomaly signals can be noisy; aggregating interactions over time reduces overreaction to transient fluctuations while still allowing persistent attackers to be separated.

4.3.1. Opinions and Evidence

For client i at round t, we represent trust as a subjective opinion

$$\chi_i^t = (b_i^t, d_i^t, u_i^t),$$

where all components are non-negative. The opinion is parameterized by non-negative evidence counts α_i^t (supporting honest behaviour) and β_i^t (supporting malicious behaviour), together with a successful packet transmission rate $q_i^t \in (0, 1)$. Setting $u_i^t = 1 - q_i^t$, the belief and disbelief masses are

$$b_i^t = (1 - u_i^t)\,\frac{\alpha_i^t}{\alpha_i^t + \beta_i^t}, \tag{13}$$

$$d_i^t = (1 - u_i^t)\,\frac{\beta_i^t}{\alpha_i^t + \beta_i^t}. \tag{14}$$

We assume in this work the communication to be stable (i.e., $u_i^t = 0$) with a successful packet transmission rate (i.e., $q_i^t = 1$). This isolates the security mechanism to data/label integrity effects rather than conflating them with link unreliability, and the same formulation readily extends when $q_i^t < 1$ by letting uncertainty absorb missing evidence. To capture both short-term and long-term behaviour, we update evidence recursively. Let $\alpha_{i,\mathrm{c}}^t \in \{0, 1\}$ and $\beta_{i,\mathrm{c}}^t \in \{0, 1\}$ denote current-round positive and negative interaction indicators. We then update

$$\alpha_i^t = \kappa(1 - \eta)\,\alpha_{i,\mathrm{c}}^t + (1 - \kappa)(1 - \eta)\,\alpha_i^{t-1}, \tag{15}$$

$$\beta_i^t = \kappa\,\eta\,\beta_{i,\mathrm{c}}^t + (1 - \kappa)\eta\,\beta_i^{t-1}, \tag{16}$$

with $\kappa \in [0, 1]$ controlling recency and $\eta \in [0, 1]$ modulating the relative impact of negative evidence. Separating recency and negativity provides a controllable bias toward faster penalization of suspicious behaviour while preserving long-term memory of consistent clients, which is useful under intermittent or adaptive attacks.

4.3.2. Interaction from Anomaly Scores

Anomaly scores are converted into interaction evidence via a threshold τ_t in the anomaly-score space:

$$\beta_{i,\mathrm{c}}^t = \begin{cases} 1, & \text{if } \hat{\delta}_i^t > \tau_t, \\ 0, & \text{otherwise,} \end{cases} \qquad \alpha_{i,\mathrm{c}}^t = 1 - \beta_{i,\mathrm{c}}^t. \tag{17}$$

Using a binary interaction abstraction keeps the reputation update lightweight and interpretable, while τ_t controls how aggressively anomaly evidence is converted into distrust.

4.3.3. Reputation Score

From each opinion χ_i^t, AD-SFL derives a scalar reputation

$$\gamma_i^t = b_i^t + a\,u_i^t, \tag{18}$$

where $a \in [0, 1]$ is a base rate reflecting the prior belief in the client honesty. We set $a = 0.5$, corresponding to a non-informative (neutral) prior: when the uncertainty mass u_i^t is high and evidence is limited, the uncertain portion is allocated equally between belief and disbelief to avoid systematic bias toward trusting or distrusting a client in the absence of observations. The vector $\boldsymbol{\gamma}^t = (\gamma_1^t, \ldots, \gamma_N^t)$ is used for dynamic client selection.

4.4. Fisher-Based Anomaly Threshold

AD-SFL must convert the continuous, per-client activation anomaly signals $\{\hat{\delta}_i^t\}_{i=1}^N$ into binary interaction evidence (*normal* vs. *suspicious*) to drive the subjective-logic update in (17). Rather than using a fixed cutoff, AD-SFL derives a *round-adaptive* anomaly threshold τ_t that separates the low-deviation majority (presumably benign) from the high-deviation tail (potentially poisoned) according to Fisher's discriminability criterion. An adaptive threshold is important because activation statistics, and hence divergence magnitudes, shift as training progresses, so a static cutoff would either under-flag early rounds or over-flag later stabilized rounds.

4.4.1. Partitioning of Clients

At round t, each client i yields a normalized activation anomaly score $\hat{\delta}_i^t$ computed from KL divergence between the client's class-conditional smashed-activation distribution and the clean reference distribution. For a candidate threshold τ, AD-SFL partitions clients into

$$C_{\mathrm{ben}}^t(\tau) = \{i : \hat{\delta}_i^t \le \tau\}, \qquad C_{\mathrm{sus}}^t(\tau) = \{i : \hat{\delta}_i^t > \tau\},$$

where $C_{\mathrm{ben}}^t(\tau)$ is interpreted as the *activation-consistent* set and $C_{\mathrm{sus}}^t(\tau)$ as the *activation-inconsistent* set.

Let $\mu_{\mathrm{ben}}^t(\tau)$, $\mu_{\mathrm{sus}}^t(\tau)$ denote the empirical means of $\hat{\delta}_i^t$ within the two sets, and $(\sigma_{\mathrm{ben}}^t(\tau))^2$, $(\sigma_{\mathrm{sus}}^t(\tau))^2$ the corresponding variances. AD-SFL selects τ to maximize the separation between the two *client groups* induced by the anomaly scores:

$$J_t(\tau) = \frac{\left(\mu_{\mathrm{sus}}^t(\tau) - \mu_{\mathrm{ben}}^t(\tau)\right)^2}{(\sigma_{\mathrm{ben}}^t(\tau))^2 + (\sigma_{\mathrm{sus}}^t(\tau))^2}. \tag{19}$$

Maximizing $J_t(\tau)$ emphasizes thresholds that simultaneously create a large between-group gap and low within-group dispersion, matching the practical objective of isolating a high-deviation tail without fragmenting the benign majority. The numerator measures the between-set gap in deviation level, while the denominator penalizes thresholds that create noisy or internally inconsistent groups. Scanning over a discrete grid $\mathcal{T}_t$ formed from the observed $\hat{\delta}_i^t$ values (a uniform range grid), AD-SFL obtains

$$\tau_t^* = \arg\max_{\tau \in \mathcal{T}_t} J_t(\tau). \tag{20}$$

Restricting candidates to the observed score range avoids introducing an additional scale parameter and ensures τ_t remains data-driven per round. Intuitively, τ_t^* is the cutoff that most cleanly isolates the high-deviation tail in the current round.

4.4.2. Safeguards for Ambiguous Rounds

Because $\{\hat{\delta}_i^t\}$ can be unimodal in attack-free rounds, blindly applying (20) may yield an arbitrary split. These checks prevent the criterion from forcing a two-cluster interpretation when the population is intrinsically homogeneous, while the high-percentile fallback remains conservative by only reacting to extreme deviations. AD-SFL therefore accepts the Fisher split only when it indicates a meaningful tail.

Dispersion check. Let $\mathrm{CV}_t = \sigma(\hat{\boldsymbol{\delta}}^t)/(\mu(\hat{\boldsymbol{\delta}}^t) + \epsilon)$. We require $\mathrm{CV}_t \geq c_{\min}$ to ensure the anomaly scores exhibit sufficient heterogeneity to justify thresholding.

Minimum standardized gap. We require a minimum effect size between the two induced sets:

$$\Delta_t(\tau) = \frac{\mu_{\text{sus}}^t(\tau) - \mu_{\text{ben}}^t(\tau)}{\sqrt{(\sigma_{\text{ben}}^t(\tau))^2 + (\sigma_{\text{sus}}^t(\tau))^2 + \epsilon}} \geq \Delta_{\min}, \tag{21}$$

which prevents selecting thresholds that produce overlapping "benign" and "suspicious" score clouds.

If either safeguard fails for $\tau_t^\star$, AD-SFL falls back to a conservative high-percentile cutoff that only flags extreme outliers:

$$\tau_t = \text{Percentile}\left(\hat{\boldsymbol{\delta}}^t,\ p_{\text{hi}}\right), \tag{22}$$

otherwise $\tau_t = \tau_t^*$.

4.5. DRL-Based Reliable Client Selection

While τ_t operates in anomaly-score space, AD-SFL ultimately requires a reputation-space threshold ω_t (over γ) to decide which clients to accept at each round. This separation decouples short-horizon anomaly sensing from longer-horizon participation control: τ_t determines evidence updates, whereas ω_t determines optimization influence through aggregation. Selecting ω_t is challenging because client behaviour and attack intensity can vary over time, making the mapping from a chosen threshold to downstream learning outcomes (loss/accuracy/participation) stochastic and nonstationary. Because the impact of excluding clients is mediated through subsequent model updates, the decision exhibits delayed and state-dependent effects, making a sequential formulation more appropriate than one-shot threshold tuning. Prior work formulates adaptive reputation-threshold selection as a sequential decision-making problem in FL environments[22]. We model ω_t selection as a finite Markov decision process (MDP) controlled by the server. In this MDP, the *agent* is the SplitFed server, and the environment comprises the SplitFed training dynamics together with the reputation mechanism that produces $\boldsymbol{\gamma}^t$. The state-transition law is unknown because it depends on latent client behaviours and the learning trajectory; thus, we adopt a model-free reinforcement learning approach that learns directly from sampled interactions.

The control variable ω_t lies in $[0, 1]$ and, in general, must be adjusted with fine granularity to trade off (i) excluding low-reputation clients to reduce adversarial influence and (ii) retaining enough participation for stable optimization. Discretizing $[0, 1]$ into a small set of actions can restrict the policy capacity and may introduce control instability, while tabular Q-learning is infeasible with continuous/high-dimensional states [22]. Continuous control is particularly relevant here since small shifts in ω_t can change the accepted set abruptly when reputations cluster, so fine-grained actions help stabilize participation-robustness trade-offs. Therefore, we learn ω_t using Deep Deterministic Policy Gradient (DDPG), an off-policy deterministic actor–critic method designed for continuous action spaces and trained with experience replay and target networks [29, 30, 31].

4.5.1. MDP Formulation

We define the MDP as $\langle \mathcal{S}, \mathcal{A}, P, r, \gamma \rangle$, where $\mathcal{S}$ is the space of observed server-side summaries, $\mathcal{A} = [\omega_{\min}, \omega_{\max}] \subseteq [0, 1]$ is the continuous action space of reputation thresholds, $P(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \omega_t)$ is the unknown transition kernel induced by SplitFed training under the chosen acceptance set, $r_t = r(\mathbf{s}_t, \omega_t)$ is the scalar reward, and $\Gamma_{disc} \in (0, 1]$ is the discount factor. The objective is to learn a policy $\pi_\theta : \mathcal{S} \to \mathcal{A}$ that maximizes the expected discounted return $\mathbb{E}[\sum_{t=1}^{T} \Gamma_{disc}^t r_t]$. In each round, the server observes $\mathbf{s}_t$, applies ω_t, forms the accepted client set $\mathcal{H}_t = \{i : \gamma_i^t \geq \omega_t\}$, aggregates updates from $\mathcal{H}_t$, and then receives r_t and the next observation $\mathbf{s}_{t+1}$.

4.5.2. State and Action

At round t, the RL agent observes the state

$$\mathbf{s}_t = [\mu(\boldsymbol{\gamma}^t), \sigma(\boldsymbol{\gamma}^t), \mathrm{acc}_{t-1}], \tag{23}$$

where $\mu(\boldsymbol{\gamma}^t)$ and $\sigma(\boldsymbol{\gamma}^t)$ are the mean and standard deviation of reputations and acc_{t-1} is the previous-round validation accuracy. These summaries capture (i) population trust level, (ii) heterogeneity that governs how sensitive acceptance is to ω_t, and (iii) a task-progress signal, yielding a low-dimensional observation that is stable across varying N.

The actor μ_θ outputs a real-valued proposal mapped into a valid range via

$$\omega_t = \text{clip}\left(\mu_\theta(\mathbf{s}_t), \omega_{\min}, \omega_{\max}\right), \tag{24}$$

with $0 \leq \omega_{\min} < \omega_{\max} \leq 1$. Clients with $\gamma_i^t \geq \omega_t$ form the accepted set $\mathcal{H}_t$. Bounding actions prevent pathological regimes (e.g., near-total rejection) and constrain exploration to operationally meaningful thresholds.

4.5.3. Reward Design

The reward favours selections that yield low task loss while maintaining participation and penalizes degenerate choices. Let

$$\bar{\mathcal{L}}_t = \frac{1}{|\mathcal{H}_t|} \sum_{i \in \mathcal{H}_t} \mathcal{L}_i^t$$

denote the mean loss over accepted clients. Define a minimum participation ratio

$$\rho_{\min} = \max\left(0.2, \frac{1}{N}\right),$$

and $\rho_t = |\mathcal{H}_t|/N$. The reward is

$$r_t = \begin{cases} -1.0, & \text{if } |\mathcal{H}_t| = 0, \\ -\lambda_L \bar{\mathcal{L}}_t + r_{\text{bal}}(\rho_t), & \text{otherwise}, \end{cases} \tag{25}$$

where $\lambda_L > 0$ controls sensitivity to the loss term and

$$r_{\text{bal}}(\rho_t) = \begin{cases} -0.5, & \text{if } \rho_t < \rho_{\min}, \\ 0, & \text{otherwise}. \end{cases} \tag{26}$$

The loss term steers the policy toward accuracy-preserving selections, while the participation penalty discourages thresholds that trivially increase robustness by starving optimization of data.

4.5.4. Learning the Threshold Policy

Transitions $(\mathbf{s}_t, \omega_t, r_t, \mathbf{s}_{t+1})$ are stored in an experience replay buffer, enabling off-policy learning from past interactions. DDPG maintains (i) an actor μ_θ that outputs deterministic actions in the continuous threshold space and (ii) a critic Q_ϕ that evaluates state–action values. Target networks and Polyak averaging are used to stabilize bootstrapped learning [29]. This yields a thresholding policy that adapts ω_t to the evolving SplitFed training dynamics and client population, without requiring an explicit model of client behaviour or the transition kernel.

4.6. Aggregation and Reference Update

After filtering clients via activation-based anomaly detection, subjective logic reputation, and DDPG-based thresholding, the server aggregates only over the accepted clients $\mathcal{H}_t$. Let n_i denote the number of samples used by client i in round t, and let $n_{\text{total}} = \sum_{i \in \mathcal{H}_t} n_i$. Sample-proportional weights retain the standard FedAvg optimization behaviour among accepted clients, ensuring that the defense changes who contributes rather than how contributions are combined. AD-SFL uses FedAvg-style aggregation:

$$W_C^{t+1} = \sum_{i \in \mathcal{H}_t} \frac{n_i}{n_{\text{total}}} W_{C,i}^{t+1}, \tag{27}$$

$$W_S^{t+1} = \sum_{i \in \mathcal{H}_t} \frac{n_i}{n_{\text{total}}} W_{S,i}^{t+1}, \tag{28}$$

where $W_{C,i}^{t+1}$ and $W_{S,i}^{t+1}$ are the updated client-side and server-side parameters for client i at the end of round t.

To keep the KL-based detector aligned with the evolving feature space, the reference activations are refreshed after each aggregation step. Without this refresh, the detector would gradually confound natural representation drift with malicious deviation, increasing false positives as training progresses. Once W_C^{t+1} is available, the server recomputes the class-conditional reference activations for each reference point and refits KDE models, which are then used for KL-based anomaly computation in the next round.

4.7. Training Workflow

Algorithm 1 Server-side workflow in AD-SFL

Input: total rounds T
Output: updated global parameters W_C, W_S
Initialize W_S^0, W_C^0, reference dataset $\mathcal{D}_{\text{ref}}$
Initialize DDPG actor/critic, target networks and replay buffer
for each global round $t = 1, \ldots, T$ **do**
  Broadcast W_C^t to all clients
  Receive smashed activations $\mathbf{a}_{i,t}$ and labels Y_i^t from all clients C_i
  for each client C_i **do**
    Server forward pass through $f_S(\cdot; W_S^t)$
    Compute loss $\mathcal{L}_i^t$
    Compute anomaly score δ_i^t and normalized score $\hat{\delta}_i^t$
  end for
  Compute Fisher-based anomaly threshold τ_t from $\{\hat{\delta}_i^t\}$
  for each client C_i **do**
    Compute opinion (b_i^t, d_i^t, u_i^t) and reputation γ_i^t
  end for
  Form state $\mathbf{s}_t$ and obtain ω_t from the actor
  Define accepted set $\mathcal{H}_t = \{i : \gamma_i^t \geq \omega_t\}$
  for each $i \in \mathcal{H}_t$ **do**
    Send cut-layer gradients $\partial \mathcal{L}_i^t / \partial \mathbf{a}_{i,t}$ to client C_i
    Receive updated client-side weights $W_{C,i}^{t+1}$ and obtain $W_{S,i}^{t+1}$ by backpropagation
  end for
  Aggregate $\{W_{C,i}^{t+1}\}_{i \in \mathcal{H}_t}$ and $\{W_{S,i}^{t+1}\}_{i \in \mathcal{H}_t}$ to get W_C^{t+1}, W_S^{t+1}
  Update reference activations and KDEs using $f_C(\cdot; W_C^{t+1})$ and $\mathcal{D}_{\text{ref}}$
  Compute reward r_t and construct next state $\mathbf{s}_{t+1}$
  Store $(\mathbf{s}_t, \omega_t, r_t, \mathbf{s}_{t+1})$ in replay buffer
  Update actor/critic and target networks
end for

The end-to-end operation of AD-SFL in a SplitFed setting is summarized by the server-side and client-side workflows in Algorithms 1 and 2, respectively. Training proceeds in a synchronized global rounds, during which the server orchestrates model broadcast, activation-based anomaly analysis, reputation updates, DDPG-based threshold selection, and aggregation, while clients perform local forward and backward passes on their private data.

On the *server side*, AD-SFL initializes the server-side subnetwork $f_S(\cdot; W_S^0)$, the shared client-side subnetwork $f_C(\cdot; W_C^0)$, and the clean reference dataset $\mathcal{D}_{\text{ref}}$, from which initial reference activation distributions are computed. In parallel, the DDPG components (actor, critic, target networks, and replay buffer) are initialized following [22]. At the beginning of each global round t, the server broadcasts W_C^t to all participating clients. After receiving smashed activations $\mathbf{a}_{i,t}$ and labels Y_i^t from each client C_i, the server performs a forward pass through $f_S(\cdot; W_S^t)$, computes per-client losses $\mathcal{L}_i^t$, and derives anomaly scores δ_i^t and $\hat{\delta}_i^t$ from the KL divergence between client and reference activation densities. Using $\{\hat{\delta}_i^t\}_{i=1}^N$, the server selects an

Algorithm 2 Client-side workflow in AD-SFL

Input: local dataset $\mathcal{D}_i$, total rounds T
Output: updated client-side parameters sent to server
Initialize client-side model $f_C(\cdot; W_C^0)$ with weights from server
for each global round $t = 1, \ldots, T$ **do**
  Receive current client-side weights W_C^t from server
  Sample local mini-batch $\mathcal{B}_i^t$ from $\mathcal{D}_i$
  Compute smashed activations $\mathbf{a}_{i,t}$ via client forward pass
  Send $\mathbf{a}_{i,t}$ and labels Y_i^t to server
  Receive cut-layer gradients $\partial\mathcal{L}_i^t/\partial\mathbf{a}_{i,t}$ if $i \in \mathcal{H}_t$
  if gradients are received **then**
    Perform client-side backpropagation through f_C to obtain updated weights $W_{C,i}^{t+1}$
    Send $W_{C,i}^{t+1}$ to server
  end if
end for

anomaly threshold τ_t via the Fisher criterion with safeguards, converts scores into subjective-logic evidence, and updates reputations γ_i^t. These reputations define the state $\mathbf{s}_t$, which is fed to the DDPG actor to obtain ω_t. Clients with $\gamma_i^t \geq \omega_t$ form the accepted set $\mathcal{H}_t$.

For each client in $\mathcal{H}_t$, the server sends back cut-layer gradients $\partial\mathcal{L}_i^t/\partial\mathbf{a}_{i,t}$, receives updated client-side parameters $W_{C,i}^{t+1}$, and performs server-side backpropagation to obtain $W_{S,i}^{t+1}$. These per-client updates are then aggregated using (27)–(28) to produce W_C^{t+1} and W_S^{t+1}. The server refreshes reference activations and refits KDEs for the next round. Finally, it computes the reward r_t using (25), constructs $\mathbf{s}_{t+1}$, stores $(\mathbf{s}_t, \omega_t, r_t, \mathbf{s}_{t+1})$ in the replay buffer, and updates the DDPG networks.

On the *client side*, each client C_i starts from the client-side parameters W_C^0 received from the server. In each global round t, the client updates its local copy of W_C^t, samples a mini-batch $\mathcal{B}_i^t$ from $\mathcal{D}_i$, performs a forward pass through $f_C(\cdot; W_C^t)$ to obtain smashed activations $\mathbf{a}_{i,t}$, and sends $\mathbf{a}_{i,t}$ and labels Y_i^t to the server. If $i \in \mathcal{H}_t$, it receives cut-layer gradients $\partial\mathcal{L}_i^t/\partial\mathbf{a}_{i,t}$, backpropagates through f_C to obtain updated weights $W_{C,i}^{t+1}$, and uploads them back to the server. Clients rejected in a round do not participate in that round's backpropagation or aggregation.

4.8. Scalability

We analyze how the per-round runtime of AD-SFL scales with the number of clients N. For clarity, we treat the mini-batch size, the number of classes, the per-sample computation costs of the client- and server-side subnetworks, and the KDE-related quantities as constants.

A standard SplitFed round (without defenses) scales linearly with N, i.e., $O(N)$, dominated by client/server forward–backward passes and the associated communication of smashed activations and gradients.

AD-SFL introduces four additional components. Activation-based anomaly detection (KDE + KL) processes a constant-size activation batch per client and thus remains $O(N)$. Fisher-based adaptive thresholding sorts the per-client anomaly scores and scans candidate cut points, incurring $O(N \log N)$. Subjective-logic reputation updates are applied independently per client and remain $O(N)$. Finally, DDPG-based client selection operates on fixed-dimensional state/action vectors and fixed network architectures, yielding $O(1)$ cost per environment step for fixed architectures [29]. Overall, the per-round complexity of AD-SFL is $O(N) + O(N \log N)$, which is asymptotically close to linear scaling and strictly below quadratic regimes.

5. Performance Evaluation

In this section, we empirically evaluate AD-SFL on two standard image classification benchmarks (MNIST, CIFAR-10) under various data poisoning scenarios. We compare our approach with state-of-the-art defenses and analyze its robustness, detection performance, and adaptivity. All experiments were conducted in a Google Colab environment using an NVIDIA A100 GPU (40 GB VRAM), and all models and defenses were implemented in `PyTorch`.

5.1. Experimental Setup

5.1.1. Datasets and Models

We consider the following benchmark datasets and backbone architectures:

- **MNIST**: 70,000 grayscale images of handwritten digits (10 classes, 28×28). We adopt a LeNet-style CNN as the global model $\mathcal{M}(\cdot)$ and implement it in a split setting. In particular, the client-side subnetwork $\mathcal{M}_c(\cdot)$ comprises the first feature extractor block (Conv→Pool), which produces the smashed activations for transmission to the server. The remaining layers are hosted on the server-side subnetwork $\mathcal{M}_s(\cdot)$, including the second convolutional block and the two fully connected layers, culminating in a 10-way classifier for digit recognition.
- **CIFAR-10**: 60,000 color images (10 classes, 32×32). We adopt a ResNet-18 backbone, a widely used standard architecture for CIFAR-10. The network is decomposed into a client-side subnetwork $\mathcal{M}_c(\cdot)$ comprising the initial convolutional layers and residual blocks, and a server-side subnetwork $\mathcal{M}_s(\cdot)$ containing the remaining residual blocks and the final classification head.
- **CIFAR-100**: 60,000 color images (100 classes, 32×32) with increased label granularity compared to CIFAR-10. We adopt an ImageNet-pretrained WideResNet-50 backbone to improve feature transfer under the more challenging 100-class setting. The network is implemented in a split configuration, where the client-side subnetwork $\mathcal{M}_c(\cdot)$ comprises the early convolutional stem and initial residual stages, while the server-side subnetwork $\mathcal{M}_s(\cdot)$ hosts the remaining residual stages and the final classification head, adapted to output a 100-way classifier.

Table 2 summarizes the key dataset statistics and model architectures.

Table 2: Datasets and backbone models used in our experiments.

Dataset	#Classes	Image size	Backbone	Split layer
MNIST	10	$1 \times 28 \times 28$	LeNet-style CNN	conv2
CIFAR-10	10	$3 \times 32 \times 32$	ResNet-18	block2
CIFAR-100	100	$3 \times 32 \times 32$	WideResNet-50	block2

Table 3: SplitFed training hyperparameters.

Parameter	**Value**
#Clients N	20
Batch size B	256
Global rounds T	100
Client learning rate	10^{-3}
Server learning rate	10^{-3}
$\lvert\mathcal{D}_{\text{ref}}\rvert$ (each class)	10

5.1.2. Non-IID data distribution

To model data heterogeneity, we partition the training data across clients using a Dirichlet-based non-IID split with concentration $\alpha = 0.5$. A smaller α produces more skewed class proportions per client. Concretely, for each class, its samples are randomly distributed among the N clients according to Dirichlet-drawn proportions, and each client receives the union of its assigned samples across classes.

5.1.3. Hyperparameters

We consider N clients in the SplitFed system. The client-side and server-side learning rates, mini-batch sizes, and total number of global rounds T are summarized in Table 3. The clean reference set $\mathcal{D}_{\text{ref}}$ is sampled from the test splits of corresponding datasets. Details of building $\mathcal{D}_{\text{ref}}$ for deployments is discussed in Section 3.2.1.

AD-SFL specific hyperparameters, including the KDE bandwidth, Fisher criterion parameters, subjective-logic parameters, and DDPG parameters, are listed in Table 4.

5.2. Attack Models and Baselines

We evaluate AD-SFL under several label-flip and backdoor attacks.

5.2.1. Attack model

A fixed fraction of clients is designated as malicious, with the malicious client ratio $M_f \in \{0, 0.3, 0.5, 0.7\}$; the case $M_f = 0$ corresponds to a fully benign baseline. Malicious clients tamper with their local labels and/or data features before training, while the data of benign clients remain unchanged.

Label-flipping. For MNIST, malicious clients flip labels symmetrically between the pairs $(1, 8)$, $(2, 7)$, and $(3, 6)$. For CIFAR-10, we flip the class pairs: *automobile* $\leftrightarrow$ *ship*, *bird* $\leftrightarrow$ *horse*, and *cat* $\leftrightarrow$ *dog*. Within each local mini-batch of a malicious client, all samples whose labels belong to these pairs are relabelled to their corresponding partner class, while all remaining labels are kept intact.

Table 4: AD-SFL and DDPG hyperparameters.

Parameter	**Value**
AD-SFL (activation anomaly, reputation, and thresholds)	
KDE kernel	Gaussian
Stability constant ϵ	10^{-8}
Dispersion threshold $c_{\min}$	0.05
Min. standardized gap $\Delta_{\min}$	2
Fallback percentile p_{hi}	90
Forgetting/recency κ	0.6
Negative evidence η	0.7
Packet success rate q	1
Base rate a	0.5
Loss weight λ_L	1
DDPG threshold agent	
Discount factor Γ_{disc}	0.995
Soft update τ_{soft}	0.005
Actor learning rate	10^{-4}
Critic learning rate	10^{-3}
Mini-batch size	32
Replay buffer size	2000
Exploration noise scale	0.05
Hidden units (actor / critic)	128 / 128

Backdoor poisoning. In addition to label flipping, we consider a backdoor attack in which malicious clients implant an attacker-chosen trigger pattern I into a subset of their local inputs and assign them an attacker-chosen target label T_A. Concretely, for each malicious client, samples drawn from a set of source classes are modified by stamping a small fixed trigger patch at a predefined location, yielding triggered inputs $x \in X_I$. These triggered samples are then relabelled to T_A, while the remaining samples in the mini-batch are left unchanged. At inference time, the attack objective is that any input containing the trigger I is predicted as the target class T_A, regardless of its true label.

5.2.2. Baselines

We evaluate AD-SFL against three baselines under an identical model split. **No Defense** runs standard SplitFed without explicit poisoning mitigation, aggregating client-side updates via FedAvg at each round. **Gold Standard** is a hypothetical oracle that perfectly identifies malicious clients and deterministically excludes their updates, aggregating only benign client models; while unattainable in practice, it provides an upper bound on robustness and utility. **SafeSplit** is a Split Learning defense tailored to mitigate client-side backdoor threats. Finally, we report **AD-SFL**, which combines activation-based anomaly detection, subjective-logic reputation modeling, Fisher-based adaptive thresholding, and DDPG-based reliable client selection.

5.3. Robustness of the Global Model

Tables 5 and 7 summarize robustness on MNIST and CIFAR-10 under label-flip and backdoor attacks as the malicious-client

Table 5: Robustness of the global model under (i) label-flip and (ii) backdoor attacks on MNIST. We report final clean test accuracy (%) and attack success rate (ASR, %) as mean ± 95% CI for varying fractions of malicious clients. AD-SFL denotes our proposed method. Results are reported for both IID and Non-IID client data partitions.

			0%		30%		50%		70%	
Data Split	Attack	Method	Acc ↑	ASR ↓	Acc ↑	ASR ↓	Acc ↑	ASR ↓	Acc ↑	ASR ↓
IID	Label-flip	No Defense	98.09±0.43	0.25±0.07	94.73±0.32	2.12±0.78	64.46±1.87	52.86±0.76	38.48±0.31	97.34±1.97
		Gold Standard	98.09±0.43	0.25±0.07	97.90±0.66	0.26±1.93	97.17±0.31	0.38±1.74	96.30±1.65	0.25±3.56
		SafeSplit	99.28±0.11	0.11±0.08	99.15±0.13	0.12±0.19	98.31±0.31	0.59±0.11	38.28±0.20	99.13±0.26
		AD-SFL	97.57±0.92	1.63±1.32	97.54±0.44	0.38±0.67	97.16±1.34	1.42±1.56	96.26±3.06	1.93±1.94
	Backdoor	No Defense	98.13±0.43	0.22±0.33	97.68±1.34	100±0.00	97.43±0.76	100±0.00	95.01±1.23	100±0.00
		Gold Standard	98.13±0.43	0.22±0.33	97.85±1.75	0.13±0.92	97.22±0.74	0.44±1.34	96.40±2.39	0.60±1.65
		SafeSplit	98.32±0.15	0.05±0.05	98.21±0.23	1.33±1.57	98.18±0.10	100.00±0.00	94.91±15.44	99.99±0.05
		AD-SFL	97.55±0.99	1.45±1.70	97.71±1.53	1.43±0.34	97.13±1.45	2.21±2.56	96.27±3.78	3.34±1.34
Non-IID	Label-flip	No Defense	97.88±0.05	0.19±0.20	93.36±1.45	4.18±4.13	65.98±1.20	42.22±2.75	55.09±3.11	62.05±1.76
		Gold Standard	97.88±0.05	0.19±0.20	97.24±1.09	0.25±0.16	95.43±2.12	0.33±0.45	89.71±1.56	0.45±3.12
		SafeSplit	98.82±0.56	0.14±0.06	98.77±0.86	0.14±0.15	56.82±5.04	66.24±3.98	38.00±1.39	98.95±0.80
		AD-SFL	96.96±0.46	1.71±1.09	97.14±1.65	1.97±0.65	95.28±3.71	2.11±5.13	91.43±1.95	2.53±7.34
	Backdoor	No Defense	97.84±3.22	0.44±0.98	97.35±1.54	100±0.00	96.39±1.22	100±0.00	94.28±3.76	100±0.00
		Gold Standard	97.84±3.22	0.44±0.98	97.27±1.81	0.72±0.64	95.39±3.91	0.82±1.49	89.73±2.14	0.76±1.84
		SafeSplit	98.91±0.63	0.05±0.12	98.62±0.89	0.23±1.92	98.75±0.61	98.60±0.40	89.05±3.10	99.13±1.90
		AD-SFL	96.08±0.87	0.48±0.22	96.97±0.54	1.35±0.28	94.35±0.49	1.48±1.67	91.61±1.36	2.98±0.62

Table 6: Robustness of the global model under (i) label-flip and (ii) backdoor attacks on CIFAR-10. We report final clean test accuracy (%) and attack success rate (ASR, %) as mean ± 95% CI for varying fractions of malicious clients. AD-SFL denotes our proposed method. Results are reported for both IID and Non-IID client data partitions.

			0%		30%		50%		70%	
Data Split	Attack	Method	Acc ↑	ASR ↓	Acc ↑	ASR ↓	Acc ↑	ASR ↓	Acc ↑	ASR ↓
IID	Label-flip	No Defense	80.08±0.28	1.40±0.40	64.75±1.28	23.60±0.46	61.82±1.80	42.40±2.15	37.26±2.70	74.40±1.19
		Gold Standard	80.08±0.65	1.40±0.93	78.46±1.80	2.83±0.46	76.55±0.86	3.20±2.64	72.87±0.92	3.80±0.74
		SafeSplit	80.21±3.55	0.46±4.26	79.82 ±2.49	7.41±1.03	73.49±4.22	17.54±1.85	53.14±1.38	57.73±3.66
		AD-SFL	78.52±0.67	1.90±0.93	80.87±0.28	3.63±1.43	77.65±0.40	3.10±2.20	72.89±0.46	4.57±1.87
	Backdoor	No Defense	81.16±0.75	1.63±0.04	78.38±0.53	93.77±0.83	66.06±0.07	97.77±9.67	60.98±0.45	99.40±0.83
		Gold Standard	81.16±0.75	1.63±0.04	78.46±0.43	1.50±0.40	76.55±1.45	2.93±1.94	72.87±0.93	2.47±1.08
		SafeSplit	82.66±1.62	0.61±0.53	80.20±2.05	1.94±2.53	79.74±22.32	95.67±8.19	71.73±22.53	96.77±10.83
		AD-SFL	78.25±1.96	0.93±0.86	81.96±3.04	1.30±4.85	78.48±9.32	2.23±11.53	72.70±0.66	2.78±0.34
Non-IID	Label-flip	No Defense	76.34±0.86	2.47±0.09	68.87±2.64	47.07±1.11	46.85±0.92	59.17±20.10	35.83±1.94	67.47±0.36
		Gold Standard	76.34±0.86	2.47±0.09	71.02±0.28	2.60±2.28	70.87±0.40	3.43±0.74	52.57±0.46	3.70±1.61
		SafeSplit	75.24±6.55	8.43±8.85	70.65±1.36	12.29±8.87	64.25±3.86	24.23±4.59	38.04±11.11	67.52±9.77
		AD-SFL	71.13±1.28	4.56±1.64	73.11±0.67	6.23±0.94	70.94±0.65	6.73±0.93	50.19±0.65	5.63±0.93
	Backdoor	No Defense	76.34±0.86	1.63±0.13	70.99±1.80	97.50±2.70	65.11±0.85	97.53±0.40	59.75±0.09	99.37±1.34
		Gold Standard	76.34±0.86	1.63±0.13	70.87±0.28	1.47±1.43	68.83±0.40	2.10±2.20	52.57±0.46	2.63±0.04
		SafeSplit	76.01±5.67	1.39±2.90	71.65±7.63	85.50±1.59	67.59±10.98	60.46±3.52	59.42±5.43	79.13±8.15
		AD-SFL	73.21±0.67	0.93±2.70	72.70±1.97	1.67±0.82	68.50±1.34	4.60±0.65	55.48±1.53	9.82±1.79

fraction increases $(0\%, 30\%, 50\%, 70\%)$ for IID and Non-IID splits. We report final clean test accuracy (*Acc*, %) and attack success rate (*ASR*, %), where higher *Acc* and lower *ASR* indicate better robustness.

Across both datasets and data partitions, **No Defense** deteriorates rapidly with stronger adversarial presence. Under label-flip, accuracy collapses while *ASR* escalates (e.g., MNIST IID: 98.09% → 38.48% with *ASR* up to 97.34%; CIFAR-10 IID: 80.08% → 37.26% with *ASR* up to 74.40%). Under backdoor, the global model becomes effectively compromised for any nonzero malicious fraction, yielding near-saturated *ASR* on MNIST (often 100%) and very high *ASR* on CIFAR-10 (IID: 93.77% → 99.40%; Non-IID: 97.50% → 99.37%).

In contrast, **AD-SFL** consistently tracks the **Gold Standard** oracle, maintaining high clean accuracy while suppressing attack success across attack types and splits. Even at 70% malicious clients, AD-SFL keeps MNIST performance strong (IID label-flip: 96.26% *Acc*/1.93% *ASR*; IID backdoor: 96.27%/3.34%; Non-IID label-flip: 91.43%/2.53%; Non-IID backdoor: 91.61%/2.98%). On CIFAR-10, AD-SFL preserves accuracy and substantially reduces *ASR*, including the most challenging Non-IID backdoor regime (70% malicious: 55.48% *Acc* with 9.82% *ASR* versus No Defense at 99.37% *ASR*).

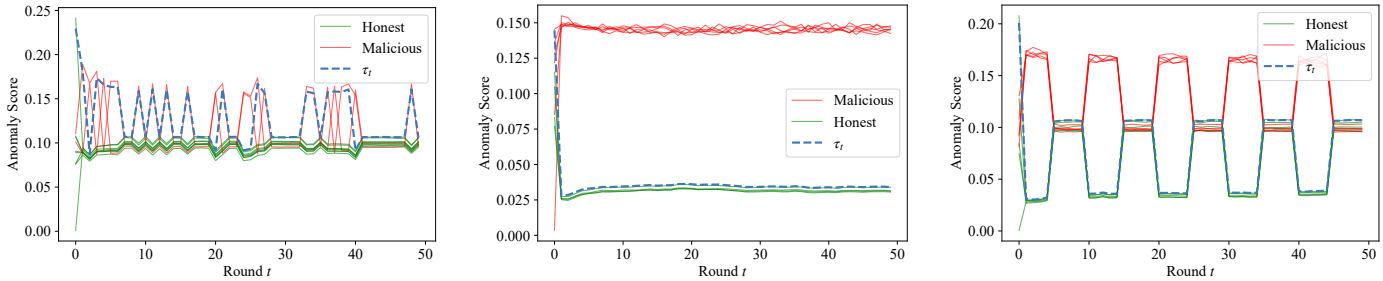
SafeSplit is less reliable under high adversarial intensity, showing pronounced breakdowns in several settings (e.g., MNIST label-flip at 70%: *Acc* ≈ 38% with *ASR* ≈ 99% in both IID and Non-IID; multiple backdoor cases with near-saturated *ASR*). Overall, the two tables indicate that AD-SFL offers the most consistent robustness across datasets, attack types, and data heterogeneity, while remaining close to oracle-level behaviour.

5.4. Adaptivity and Threshold Dynamics

We analyze how the Fisher-based anomaly threshold τ_t and the DDPG-learned reputation threshold ω_t evolve over training

Table 7: Robustness of the global model under (i) label-flip and (ii) backdoor attacks on CIFAR-100. We report final clean test accuracy (%) and attack success rate (ASR, %) as mean ± 95% CI for varying fractions of malicious clients. AD-SFL denotes our proposed method. Results are reported for both IID and Non-IID client data partitions.

			0%		30%		50%		70%	
Data Split	Attack	Method	Acc ↑	ASR ↓	Acc ↑	ASR ↓	Acc ↑	ASR ↓	Acc ↑	ASR ↓
IID	Label-flip	No Defense	68.53±1.85	0.12±1.33	−±−	−±−	−±−	−±−	47.17±2.50	18.67±2.19
		Gold Standard	68.53±1.85	0.12±1.33	−±−	−±−	−±−	−±−	51.56±2.39	1.83±0.89
		SafeSplit	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		AD-SFL	60.39±1.42	0.34±1.40	−±−	−±−	−±−	−±−	51.35±1.92	2.34±1.53
	Backdoor	No Defense	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		Gold Standard	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		SafeSplit	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		AD-SFL	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
Non-IID	Label-flip	No Defense	57.53±1.24	0.29±0.24	−±−	−±−	−±−	−±−	34.32±1.55	29.00±2.99
		Gold Standard	57.53±1.24	0.29±0.24	−±−	−±−	−±−	−±−	53.86±2.12	3.67±1.43
		SafeSplit	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		AD-SFL	53.21±1.87	1.23±1.02	−±−	−±−	−±−	−±−	49.86±2.31	5.89±1.72
	Backdoor	No Defense	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		Gold Standard	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		SafeSplit	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−
		AD-SFL	−±−	−±−	−±−	−±−	−±−	−±−	−±−	−±−



(a) Low-intensity attack (b) High-intensity attack (c) Time-varying attack

Figure 2: Evolution of client anomaly scores and Fisher-based threshold τ_t under different attack regimes. Green curves correspond to honest clients, red curves to malicious clients, and the dashed blue line shows the adaptive threshold τ_t.

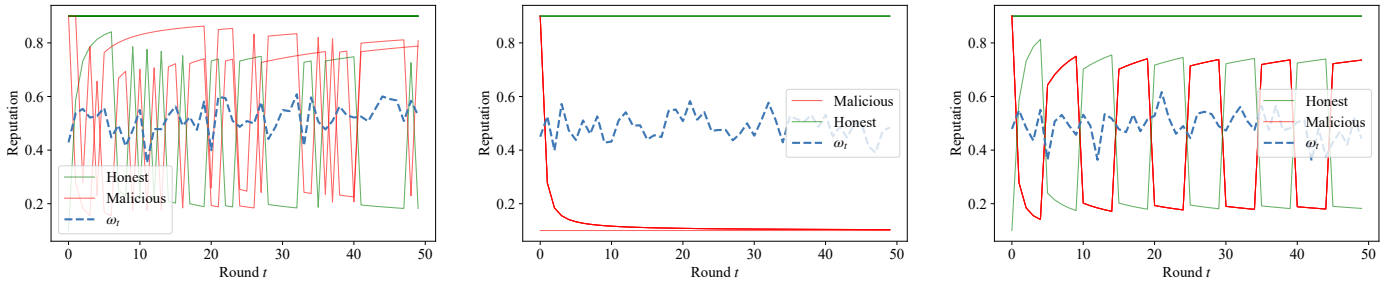
under three types of label-flip attacks in the IID setting: (i) a *low-intensity* "random" attack with 20% malicious clients, each flipping 40% of their local samples with attack probability 0.3 per round; (ii) a *high-intensity* "always-on" attack with 60% malicious clients, flipping 80% of samples; and (iii) a *time-varying* "cyclic" attack with 50% malicious clients, each attacking periodically while flipping 80% of the targeted samples.

5.4.1. *Fisher-based Anomaly Threshold τ_t*

Figures 2a–2c plot the evolution of the per-round anomaly scores and the corresponding Fisher-derived threshold τ_t for the three attack settings. In all cases, honest clients (green) exhibit consistently lower anomaly scores than malicious ones (red) after a short warm-up period, and τ_t automatically adapts to the separation between the two groups.

Low-intensity Random Attack. In the low-attack regime (Fig. 2a), the gap between honest and malicious anomaly scores is relatively small and fluctuates from round to round, since only 20% of the clients are malicious and each attacks with probability 0.3. As a result, the distribution of $\{\hat{\delta}_i^t\}$ is frequently close to unimodal, and the KDE-based KL estimation can struggle to reliably separate slightly deviating clients from the honest majority. In such rounds, the Fisher criterion may fail the coefficient-of-variation and separation checks, triggering the fallback percentile rule. This produces the seemingly random spikes in τ_t: the threshold is temporarily pushed up to the upper tail of the score distribution, effectively excluding at most one marginal client while still penalizing the most deviating one. Over time, these fallback-induced spikes lead to small but noticeable oscillations of τ_t around its steady level, reflecting a conservative behaviour that limits false positives under low attack intensity yet remains sensitive to occasional stronger deviations.

High-intensity Always-on Attack. Under the high-attack configuration (Fig. 2b), most clients (60%) behave maliciously in almost every round and flip 80% of their targeted samples. After an initial transient (first 2–3 rounds), the anomaly scores clearly split into two well-separated clusters, with malicious clients around a high plateau (approximately 0.14) and honest clients around a much lower plateau (approximately 0.03). The



(a) Low-intensity attack (b) High-intensity attack (c) Time-varying attack

Figure 3: Evolution of client reputations $\gamma_{i,t}$ and the DDPG-learned reputation threshold ω_t across different attack regimes. Green curves correspond to honest clients, red curves to malicious clients, and the dashed blue line shows the adaptive threshold ω_t.

increased between-class variance and relatively small within-class variance cause the Fisher criterion to select a stable threshold in the gap between the clusters. As a result, τ_t rapidly drops from its initial value and settles close to the honest plateau, remaining nearly constant for the rest of training. This behaviour indicates that, in strongly adversarial environments, the Fisher-based mechanism quickly locks onto a discriminative threshold that reliably isolates malicious clients.

Time-varying Cyclic Attack. In the time-varying scenario (Fig. 2c), malicious clients attack only during specific phases and revert to benign behaviour in the remaining rounds. This produces a sawtooth pattern in their anomaly scores, which alternate between a high cluster (during active attack phases) and a low cluster that overlaps with honest clients (during inactive phases). The threshold τ_t mirrors this periodic structure. When the attack is active and the two clusters are well separated, the Fisher criterion identifies a large between-class gap and moves τ_t downward, separating the honest and malicious plateaus. When the attack switches off and the score distribution becomes essentially unimodal, the safeguards are triggered: the coefficient of variation drops below $c_{\max}$, the standardized separation shrinks, and AD-SFL reverts to the percentile-based rule. In these rounds, τ_t rises to near the bulk of honest scores. Overall, AD-SFL makes τ_t dynamically track the current score distribution, reacting quickly to both the onset and disappearance of malicious activity.

5.4.2. Reputation Threshold ω_t

Figures 2a–2c show how the reputations γ_i^t and the DDPG-learned reputation threshold ω_t evolve under the three attack settings. Reputations integrate anomaly-based evidence across rounds with an asymmetric update that discounts past behaviour and gives more weight to negative interactions, while the actor observes summary statistics of $\boldsymbol{\gamma}^t$, the previous validation accuracy, and the normalized time step. The learned policy therefore adjusts ω_t so as to keep malicious clients mostly below the threshold and honest clients above it, while still maintaining a sufficient participation ratio.

Low-intensity Random Attack. In the low-attack scenario (Fig. 3a), most honest clients quickly accumulate positive evidence and their reputations rise towards a stable plateau close to 0.9. Malicious clients, on the other hand, attack only with probability 0.3 and therefore alternate between benign and harmful behaviour. This interplay with the forgetting factor κ and the negative-evidence weight η leads to strongly fluctuating reputations for malicious clients, which jump up in rounds where they behave honestly and drop sharply whenever their anomaly score exceeds τ_t. A single honest client also shows noticeable reputation dips: when the Fisher test on $\{\hat{\delta}_i^t\}$ fails and the fallback percentile rule produces a spike in τ_t, this borderline honest client is occasionally flagged as anomalous and temporarily accumulates negative evidence, before its reputation recovers in subsequent benign rounds. The learned threshold ω_t settles in the mid-range (roughly 0.45–0.6), with visible jitter caused by DDPG exploration and the stochastic mixture of honest/malicious updates. Even when ω_t moves up and down, the main honest plateau remains safely above the threshold, whereas most malicious clients stay below it except for occasional short-lived excursions when their recent behaviour appears benign.

High-intensity Always-on Attack. Under the high-attack configuration (Fig. 3b), malicious clients attack in almost every round, so they rapidly accumulate negative evidence and their reputations decay towards a very low plateau (below 0.1). Honest clients again climb monotonically to a high value near 0.9. The clear and persistent separation between the two groups makes the control problem easier: after a brief exploratory phase, the DDPG agent stabilizes ω_t around the middle of the gap (approximately 0.45–0.55) and only exhibits small fluctuations due to exploration. With this choice, essentially all honest clients are retained while all malicious clients are consistently rejected in later rounds, matching the behaviour observed in the robustness tables.

Time-varying Cyclic Attack. In the time-varying case (Fig. 3c), malicious clients attack only during specific phases and remain benign in the others. This produces a characteristic sawtooth pattern in their reputations: as soon as an attack phase starts, repeated negative evidence drives the reputations down; when the attack is switched off, positive evidence slowly rebuilds trust, but never enough to reach the honest plateau. Most honest clients stay close to 0.9 throughout, although one border-

line honest client exhibits mild cyclic dips: when the attack phase begins and the Fisher test momentarily fails, the fallback-induced spike in τ_t can briefly classify this client as anomalous, causing short drops in its reputation before it recovers in subsequent benign rounds. The threshold ω_t adapts smoothly to this periodic environment: it oscillates within a narrow band in the middle of the reputation range, slightly lowering itself when many clients appear benign (conserving participation) and moving upward once repeated attacks make the gap between honest and malicious clients more pronounced. Overall, the learned policy does not collapse to a trivial "always accept" or "always reject" strategy; instead, it tracks the evolving reputation landscape and maintains a stable separation between honest and malicious clients across different attack regimes, while tolerating a small number of temporary misclassifications of borderline honest clients.

5.4.3. Reward Dynamics

Figure 4 plots the evolution of the per-round reward r_t in (25) for the three attack regimes. At the beginning of training ($t \approx 1$–2), the actor has not yet learned a meaningful threshold, so ω_t often either rejects too many clients (violating the minimum participation constraint) or accepts many malicious ones, leading to large negative rewards around -1.1. As DDPG updates the policy, r_t rapidly improves: by about $t = 10$ all three curves have moved into the $[-0.2, -0.1]$ range, indicating that the agent has learned to keep a sufficiently large accepted set while reducing the average loss $\bar{\mathcal{L}}_t$ of those clients. After this transient phase, the reward continues to increase slowly and approaches values close to 0 by the end of training.

The low-intensity and time-varying attacks yield slightly higher rewards than the high-intensity regime throughout training, reflecting the fact that it is easier for the policy to find a threshold that simultaneously filters malicious clients and preserves accuracy when attack pressure is lower or intermittent. Nevertheless, the gap between the three curves is small, showing that the learned policy remains stable and effective across all attack patterns: once converged, ω_t delivers near-maximal reward even when a large majority of clients behave adversarially.

5.5. Comparison with Other Defenses

We further compare AD-SFL against representative robust aggregation and split-learning defenses under a *CIFAR-10 backdoor* setting with **20% malicious clients** and a **IID degree of 0.8** (Table 8). We report *clean test accuracy* (Acc) and *attack success rate* (ASR), where lower ASR indicates stronger backdoor mitigation.

In addition to **No Defense** and the oracle **Gold Standard**, we include two classical baselines that we *adapt to SplitFed*: (i) **KRUM** as a Byzantine-robust aggregation rule [4], and (ii) **Differential Privacy (DP)** as an update-perturbation baseline [21]. We also include two *U-shaped split learning* defenses, **SafeSplit** [13] and **ZORRO** [14].

Table 8 shows that **AD-SFL** achieves a strong robustness-utility trade-off in this Non-IID regime: it improves over **No Defense** by reducing ASR from **97.50%** to **1.78%** while also increasing clean accuracy from **70.99%** to **73.67%**. Compared to the oracle **Gold Standard** (Acc **73.75%**, ASR **1.49%**), AD-SFL remains essentially matched in clean accuracy and achieves *lower* ASR in this setting. Relative to robust aggregation baselines, **KRUM** and **DP** substantially reduce ASR versus No Defense (to **46.80%** and **50.38%**, respectively) but incur a pronounced accuracy drop (to **63.09%** and **65.70%**), indicating a weaker robustness–utility balance under split learning and heterogeneous data.

ZORRO achieves higher accuracy (**73.51%**) with low ASR (**3.16%**), while **AD-SFL** further improves ASR (**1.78%**) at comparable accuracy (**73.67%**). Overall, these results suggest that AD-SFL can approach oracle-like behavior in Non-IID split learning while preserving clean performance, without requiring perfect knowledge of malicious clients.

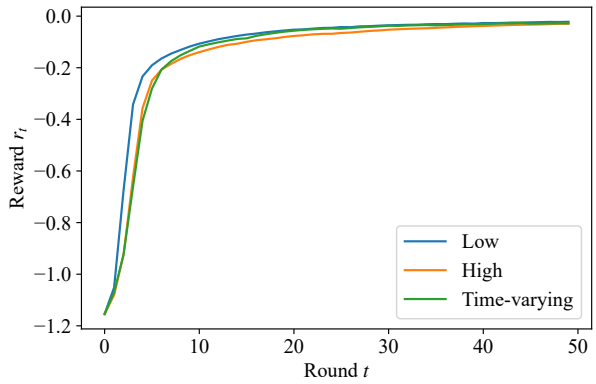


Figure 4: Evolution of the per-round reward r_t for low, high, and time-varying attack regimes. Higher values (closer to 0) indicate a better trade-off between low loss and sufficient client participation.

Table 8: Comparison with other defenses under **CIFAR-10 Non-IID (degree 0.8)** backdoor attack with 20% malicious clients.

Method	Non-IID (Acc↑, ASR↓)	
	Acc	ASR
No Defense	70.99	97.50
Gold Standard	73.75	1.49
KRUM	63.09	46.80
DP	65.70	50.38
ZORRO	73.51	3.16
AD-SFL	73.67	1.78

5.6. Ablation Study

We evaluate the sensitivity of the proposed framework to its key thresholding mechanisms via a controlled ablation study. All experiments are conducted with N=10 clients, a malicious-client ratio of 0.4, and label-flip poisoning. Unless stated otherwise, the fixed thresholds are set to τ=0.1 and ω=0.7. The SplitFed protocol, data partition, attack configuration, and all remaining hyperparameters are kept identical across variants.

The baseline is highly vulnerable to label-flip poisoning, ending at **Acc** = 0.6717 with **ASR** = 0.4072, indicating that naive aggregation allows malicious clients to simultaneously

Variant	Acc ↑	ASR ↓
Base (undefended)	0.6717	0.4072
KL screening (fixed τ)	0.9603	0.0060
KL screening (adaptive τ_t)	0.8505	0.1305
KL + Reputation (fixed τ, fixed ω)	0.9556	0.0044
KL + Fisher-τ_t + Reputation (fixed ω)	0.9693	0.0078
Full method (RL-selected ω_t)	0.9523	0.0580

Table 9: Ablation results under N=10 clients, malicious ratio 0.4, and label-flip poisoning. Fixed thresholds use τ=0.1 and ω=0.7. Higher Acc and lower ASR indicate better robustness.

degrade utility and sustain a high attack success rate. Using activation-based KL screening with a fixed cutoff (τ=0.1) yields a substantial robustness gain, achieving **Acc** = 0.9603 and **ASR** = 0.0060. This suggests that, in this experimental regime, the activation-space anomaly signal is strongly separable and a well-chosen static cutoff is sufficient to filter malicious contributions.

Replacing the fixed cutoff with an adaptive cutoff (τ_t) reduces robustness (**Acc** = 0.8505, **ASR** = 0.1305). This outcome is consistent with per-round threshold adaptation being sensitive to irregular or weakly bimodal anomaly-score distributions, which can cause unstable filtering decisions under label-flip poisoning. Adding temporal reputation accumulation while keeping thresholds fixed (τ=0.1, ω=0.7) produces the lowest ASR in Table 9 (**ASR** = 0.0044) with high clean accuracy (**Acc** = 0.9556). This indicates that reputation effectively smooths transient, round-specific screening errors by aggregating evidence over time.

Combining adaptive anomaly cutoff selection with reputation remains highly robust (**Acc** = 0.9693, **ASR** = 0.0078). Compared with adaptive τ_t alone, the improvement highlights that temporal reputation stabilizes cutoff drift, making adaptive screening more reliable in practice. Enabling RL-based admission control underperforms the strongest fixed-threshold variants at convergence (**Acc** = 0.9523, **ASR** = 0.0580). This suggests that, under the current training regime, the learned policy for ω_t is not yet consistently selecting an admission threshold that minimizes ASR, and may require additional calibration (e.g., reward scaling, exploration schedule, or training horizon).

Fixed thresholds (τ=0.1, ω=0.7) deliver excellent robustness here and can reduce server-side overhead by eliminating policy learning and simplifying admission control. However, fixed settings are inherently less adaptive and may not generalize when the attack pattern, periodicity, or intensity changes, where adaptive thresholding and learned control are expected to offer greater resilience as demonstrated in Sec. 5.4.

5.7. Communication and Computation Overhead

Table 10 reports end-to-end, per-round computation and communication overhead for SplitFed and AD-SFL (including defense, training, and evaluation). AD-SFL preserves the SplitFed protocol and centralizes the additional detection and thresholding at the server, without introducing extra client-side messages beyond the standard smashed-activation and loss exchanges.

Table 10: Average per-round overhead of SplitFed vs. AD-SFL.

Metric	SplitFed	AD-SFL
Computation (seconds per round)		
Client (avg per client)	0.0551	0.0622
Server (total)	0.9982	89.1111
Communication (MB per round)		
Client (avg per client)	39.56	26.33
Server (total)	395.60	263.33

Computation overhead. AD-SFL increases server compute (89.11 s vs. 1.00 s per round), mainly due to activation anomaly scoring via KDE-estimated KL divergence. KDE requires repeated kernel density evaluations over class-conditional smashed-activation samples against a clean reference distribution, which is substantially more expensive than histogram-based scoring or running no detector. Client compute changes only slightly (0.0622 s vs. 0.0551 s per round per client) since the additional logic is executed centrally.

Communication overhead. AD-SFL reduces communication burden, as client traffic drops from 39.56 MB to 26.33 MB per round (per-client average) and server traffic from 395.60 MB to 263.33 MB per round. The reduction follows from selective participation, as only accepted clients proceed with cut-layer gradient exchange and client-model upload. In contrast, rejected clients are skipped for that round, thereby reducing both activation/gradient traffic, as well as uploaded updates.

Overall, AD-SFL trades higher server computation for lower bandwidth usage by filtering suspicious clients and avoiding communication on potentially poisoned contributions.

6. Conclusion

In this work, we presented AD-SFL, an activation-space defense framework designed to mitigate data poisoning attacks in SplitFed learning. Instead of altering the underlying SplitFed architecture, AD-SFL leveraged the already exposed smashed activations to characterize client behavior. By comparing these against a small, trusted reference dataset, the framework measured class-conditional distribution divergences and translated them into privacy-preserving anomaly scores.

Building upon these scores, AD-SFL integrated subjective logic-based reputation modeling with a continuous-control DDPG agent. This combination allowed the server to fuse short-term anomaly evidence with long-term behavioral history to dynamically adjust the acceptance threshold. Consequently, the framework selectively filtered out malicious updates while maintaining optimal client participation, all while preserving linear scaling with the number of clients and adding minimal computational overhead.

Extensive evaluations on MNIST and CIFAR-10 demonstrated that AD-SFL substantially improved global model robustness under diverse attack configurations. Because the defense dynamically adapts to activation-space shifts and effec-

tively isolates high-deviation clients, it successfully maintained up to 96.26% clean accuracy while suppressing the attack success rate to 1.93% on MNIST, and achieved 72.70% accuracy with a 1.67% attack success rate on CIFAR-10. Additional investigation may further add insights related to the framework's applicability in highly constrained or cross-silo environments. To advance this, future research will explore lightweight parametric density models to reduce computational overhead, investigate robust calibration methods for data-scarce domains lacking pristine reference data, and integrate complementary secure computation mechanisms to relax the honest-server assumption and mitigate potential activation leakage.

Acknowledgments

Thanks to ...

References

[1] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, Jan. 2019.

[2] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," in *AAAI Conference on Artificial Intelligence*, vol. 33, New Orleans, Louisiana, USA, 2018, pp. 5895–5901.

[3] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "Splitfed: When federated learning meets split learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8485–8493.

[4] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, California, USA, 2017.

[5] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 80. Stockholm, Sweden: PMLR, 2018, pp. 5650–5659.

[6] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*. San Sebastian: USENIX Association, Oct. 2020, pp. 301–316.

[7] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 108. PMLR, 2020, pp. 2938–2948.

[8] X. Zhou, X. Chen, S. Liu, X. Fan, Q. Sun, L. Chen, M. Qiu, and T. Xiang, "Flguardian: Defending against model poisoning attacks via fine-grained detection in federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 5396–5410, 2025.

[9] J. Liu, X. Li, X. Liu, H. Zhang, Y. Miao, and R. H. Deng, "Defendfl: A privacy-preserving federated learning scheme against poisoning attacks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 5, pp. 9098–9111, 2025.

[10] J. Lu, S. Hu, W. Wan, M. Li, L. Y. Zhang, L. Xue, and H. Jin, "Depriving the survival space of adversaries against poisoned gradients in federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5405–5418, 2024.

[11] H. Luo, Y. Chen, D. Lan, H. Dou, K. Xiong, and C. Wu, "Fedcoguard: A defense framework for federated learning against untargeted poisoning attacks in sustainable agricultural iot," *IEEE Internet of Things Journal*, vol. 13, no. 1, pp. 1475–1489, 2026.

[12] Z. Wang, Z. Zhang, Z. Li, Y. Wu, Y. Liu, M. Li, X. Li, Y. Liu, J. An, W. Liang, and L. Zhu, "Resisting poisoning attacks in federated learning via dual-domain distance and trust assessment," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 7394–7409, 2025.

[13] P. Rieger, A. Pegoraro, K. Kumari, T. Abera, J. Knauer, and A.-R. Sadeghi, "Safesplit: A novel defense against client-side backdoor attacks in split learning," in *32nd Annual Network and Distributed System Security Symposium, NDSS*, 2025.

[14] N. Sheybani, A. Pegoraro, J. Knauer, P. Rieger, E. Mollakuqe, F. Koushanfar, and A.-R. Sadeghi, "Zorro: Zero-knowledge robustness and privacy for split learning," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, Taipei, Taiwn, 2025, pp. 321–334.

[15] H. Wang, Z. Yin, B. Chen, Y. Zeng, X. Yan, C. Zhou, and A. Li, "Rofed-llm: Robust federated learning for large language models in adversarial wireless environments," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 1084–1096, 2026.

[16] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *International Conference on Machine Learning*, California, USA, 2019, pp. 634–643.

[17] G. Wang, H. Wu, Q. Li, Z. Liu, J. Zhao, K.-Y. Lam, Q. Dong, Y. Yang, and L. Zhou, "Pdfl: A privacy-enhancing and robust poisoning defense federated learning scheme," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 608–623, 2026.

[18] Y. He, Z. Shen, J. Hua, Q. Dong, J. Niu, W. Tong, X. Huang, C. Li, and S. Zhong, "Backdoor attack against split neural network-based vertical federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 748–763, 2024.

[19] X. Xiao, Z. Tang, C. Li, B. Jiang, and K. Li, "Sbpa: Sybil-based backdoor poisoning attacks for distributed big data in aiot-based federated learning system," *IEEE Transactions on Big Data*, vol. 10, no. 6, pp. 827–838, 2024.

[20] X. Shen, Y. Liu, F. Li, and C. Li, "Privacy-preserving federated learning against label-flipping attacks on non-iid data," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1241–1255, 2024.

[21] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Vienna, Austria: ACM, 2016.

[22] N. M. Al-Maslamani, B. S. Ciftler, M. Abdallah, and M. M. E. A. Mahmoud, "Toward secure federated learning for iot using DRL-enabled reputation mechanism," *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 21 971–21 983, 2022.

[23] X. Zhu, J. Qi, J. Hua, T. Li, L. Yang, Z. Dong, and Y. Sun, "Pflse: A personalized federated learning framework based on shannon entropy metric for intrusion detection in iiot," *IEEE Internet of Things Journal*, vol. 13, no. 2, pp. 2936–2955, 2026.

[24] J. Kang, Z. Xiong, D. Niyato, S. Xie, and J. Zhang, "Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory," *IEEE Internet of Things Journal*, vol. 6, no. 6, pp. 10 700–10 714, 2019.

[25] J. Li, A. S. Rakin, X. Chen, Z. He, D. Fan, and C. Chakrabarti, "Ressfl: A resistance transfer framework for defending model inversion attack in split federated learning," in *Advances in Neural Information Processing Systems (NeurIPS) Workshops*, 2022.

[26] X. Yao *et al.*, "Towards privacy-preserving split learning for controlnet," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2025.

[27] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*. Chapman and Hall, 1986.

[28] P. Roy, S. Sarker, M. A. Razzaque, and G. Fortino, "Distributed task allocation in mobile device cloud exploiting federated learning and subjective logic," *Journal of Systems Architecture*, vol. 113, p. 101972, 2021.

[29] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[30] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014, pp. 387–395.

[31] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.